

注：如无特殊说明，对于图 $G = (V, E)$ ， n 指 $|V|$ ， m 指 $|E|$ 。

图论

图论相关概念

图

图是一个二元组 $G = (V(G), E(G))$ ，其中 G 代表 **图 (graph)**， V 代表 **顶点 (vertices)**， E 代表 **边 (edge)**。点数 $|V(G)|$ 也称作**阶 (order)** 数。

有向图： $G = (V, E)$ ，其中边集 $E(G)$ 仅包含有向边。

无向图： $G = (V, E)$ ，其中边集 $E(G)$ 仅包含无向边。

混合图： $G = (V, E)$ ，其中边集 $E(G)$ 既包含无向边，也包含有向边。

相邻

对于无向图 $G = (V, E)$ ，若点 v 是边 e 的一个端点，则称 v 和 e 是 **相邻的 (adjacent)**。对于两顶点 u 和 v ，若存在边 (u, v) ，则称 u 和 v 是 **相邻的 (adjacent)**。

一个顶点 $v \in V$ 的 **邻域 (neighborhood)** 是所有与之相邻的顶点所构成的集合，记作 $N(v)$ 。

一个点集 S 的邻域是所有与 S 中至少一个点相邻的点所构成的集合，记作 $N(S)$ ，即：
 $N(s) = \cup_{v \in S} N(v)$ 。

简单图

自环 (loop)：对 E 中的边 $e = (u, v)$ ，若 $u = v$ ，则 e 被称作一个自环。

重边 (multiple edge)：若 E 中存在两个完全相同的元素（边） e_1, e_2 ，则它们被称作（一组）重边。

简单图 (simple graph)：若一个图中没有自环和重边，它被称为简单图。具有至少两个顶点的简单无向图中一定存在度相同的结点。

如果一张图中有自环或重边，则称它为 **多重图 (multigraph)**。

度数

与一个顶点 v 相邻的边的条数称作该顶点的 **度 (degree)**，记作 $d(v)$ 。特别地，对于边 (v, v) ，则每条这样的边要对 $d(v)$ 产生 2 的贡献。

对于无向简单图，有 $d(v) = |N(v)|$ 。

握手定理：对于任何无向图 $G = (V, E)$ ，有 $\sum_{v \in V} d(v) = 2 \times |E|$ 。

推论：在任意图中，度数为奇数的点必然有偶数个。

若 $d(v) = 0$ ，则称为 **孤立点 (isolated vertex)**。

若 $d(v) = 1$ ，则称为 **叶节点 (leaf vertex)**。

若 $d(v) \bmod 2 = 0$ ，则称为 **偶点 (even vertex)**。

若 $d(v) \bmod 2 = 1$ ，则称为 **奇点 (odd vertex)**。图中奇点的个数是偶数。

若 $d(v) = |V| - 1$ ，则称为 **支配点 (universal vertex)**。

对一张图，所有节点的度数的最小值称为 G 的 **最小度 (minimum degree)**，记作 $\delta(G)$ ；最大值称为 G 的 **最大度 (maximum degree)**，记作 $\Delta(G)$ 。

在有向图 $G = (V, E)$ 中，以一个顶点 v 为起点的边的条数称为该顶点的 **出度 (out-degree)**，记作 $d^+(v)$ 。以一个顶点 v 为终点的边的条数称为该节点的 **入度 (in-degree)**，记作 $d^-(v)$ 。显然 $d^+(v) + d^-(v) = d(v)$ 。

对于任何有向图 $G = (V, E)$ ，有：

$$\sum_{v \in V} d^+(v) = \sum_{v \in V} d^-(v) = |E|$$

若对一张无向图 $G = (V, E)$ ，每个顶点的度数都是一个固定的常数 k ，则称为 **k-正则图 (k-regular graph)**。

路径

途径 (walk)：途径是连接一连串顶点的边的序列，可以为有限或无限长度。形式化地说，一条有限途径 w 是一个边的序列 $e_1, e_2, \dots, e_k$ ，使得存在一个顶点序列 $v_0, v_1, \dots, v_k$ 满足 $e_i = (v_{i-1}, v_i)$ ， $i \in [1, k]$ 。这样的途径可以简写为 $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ 。通常来说，边的数量 k 被称作这条途径的 **长度**（如果边是带权的，长度通常指途径上的边权之和，题目中也可能另有定义）。

迹 (trail)：对于一条途径 w ，若 $e_1, e_2, \dots, e_k$ 两两不同，则称 w 是一条迹。

路径 (path)（又称 **简单路径 (simple path)**）：对于一条迹 w ，若其连接的点的序列中点两两不同，则称 w 是一条路径。

回路 (circuit)：对于一条迹 w ，若 $v_0 = v_k$ ，则称 w 是一条回路。

环/圈 (cycle)（又称 **简单回路/简单环 (simple circuit)**）：对于一条回路 w ，若 $v_0 = v_k$ 是点序列中唯一重复出现的点对，则称 w 是一个环。

子图

对一张图 $G = (V, E)$ ，若存在另一张图 $H = (V', E')$ 满足 $V' \subseteq V$ 且 $E' \subseteq E$ ，则称 H 是 G 的 **子图 (subgraph)**，记作 $H \subseteq G$ 。

若对 $H \subseteq G$ ，满足 $\forall u, v \in V'$ ，只要 $(u, v) \in E$ ，均有 $(u, v) \in E'$ ，则称 H 是 G 的 **导出子图/诱导子图 (induced subgraph)**。

容易发现，一个图的导出子图仅由子图的点集决定，因此点集为 V' ($V' \subseteq V$) 的导出子图称为 V' 导出的子图，记作 $G[V']$ 。

若 $H \subseteq G$ 满足 $V' = V$ ，则称 H 为 G 的 **生成子图/支撑子图 (spanning subgraph)**。

显然， G 是自身的子图，支撑子图，导出子图；无边图是 G 的支撑子图。

如果一张无向图 G 的某个生成子图 F 为 k -正则图，则称 F 为 G 的一个 **k-因子 (k-factor)**。

如果有向图 $G = (V, E)$ 的导出子图 $H = G[V^*]$ 满足 $\forall v \in V^*, (v, u) \in E$ ，有 $u \in V^*$ ，则称 H 为 G 的一个 **闭合子图 (closed subgraph)**。

联通

无向图

对于一张无向图 $G = (V, E)$ ，对于 $u, v \in V$ ，若存在一条途径使得 $v_0 = u, v_k = v$ ，则称 u 和 v 是 **连通的 (connected)**。由定义，任意一个顶点和自身连通，任意一条边的两个端点连通。

若无向图 $G = (V, E)$ ，满足其中任意两个顶点均连通，则称 G 是 **连通图 (connected graph)**， G 的这一性质称作 **连通性 (connectivity)**。

若 H 是 G 的一个连通子图，且不存在 F 满足 $H \subsetneq F \subseteq G$ 且 F 为连通图，则 H 是 G 的一个 **连通块/连通分量 (connected component)** (极大连通子图)。

有向图

对于一张有向图 $G = (V, E)$ ，对于 $u, v \in V$ ，若存在一条途径使得 $v_0 = u, v_k = v$ ，则称 u **可达** v 。由定义，任意一个顶点可达自身，任意一条边的起点可达终点。(无向图中的连通也可以视作双向可达。)

若一张有向图的节点两两互相可达，则称这张图是 **强连通的 (strongly connected)**。

若一张有向图的边替换为无向边后可以得到一张连通图，则称原来这张有向图是 **弱连通的 (weakly connected)**。

与连通分量类似，也有 **弱连通分量 (weakly connected component)** (极大弱连通子图) 和 **强连通分量 (strongly connected component)** (极大强连通子图)。

割

对于连通图 $G = (V, E)$ ，若 $V' \subseteq V$ 且 $G[V \setminus V']$ (即从 G 中删去 V' 中的点) 不是连通图，则 V' 是图 G 的一个 **点割集 (vertex cut/separating set)**。大小为一的点割集又被称作 **割点 (cut vertex)**。

对于连通图 $G = (V, E)$ 和整数 k ，若 $|V| \geq k + 1$ 且 G 不存在大小为 $k - 1$ 的点割集，则称图是 **k -点连通的 (k -vertex-connected)**，而使得上式成立的最大的 k 被称作图 G 的 **点连通度 (vertex connectivity)**，记作 $\kappa(G)$ 。(对于非完全图，点连通度即为最小点割集的大小，而完全图 K_n 的点连通度为 $n - 1$ 。)

还可以在边上作类似的定义：

对于连通图 $G = (V, E)$ ，若 $E' \subseteq E$ 且 $G' = (V, E \setminus E')$ (即从 G 中删去 E' 中的边) 不是连通图，则 E' 是图 G 的一个 **边割集 (edge cut)**。大小为一的边割集又被称作 **桥 (bridge)**。

对于连通图 $G = (V, E)$ 和整数 k ，若不存在大小为 k 的边割集，则称图 G 是 **k -边连通的 (k -edge-connected)**，而使得上式成立的最大的 k 被称作图 G 的 **边连通度 (edge connectivity)**，记作 $\lambda(G)$ 。(对于任何图，边连通度即为最小边割集的大小。)

点双连通 (biconnected) 几乎与 2-点连通完全一致，除了一条边连接两个点构成的图，它是点双连通的，但不是 2-点连通的。换句话说，没有割点的连通图是点双连通的。

边双连通 (-edge-connected) 与 2-边双连通完全一致。换句话说，没有桥的连通图是边双连通的。

与连通分量类似，也有 **点双连通分量 (biconnected component)** (极大点双连通子图) 和 **边双连通分量 (-edge-connected component)** (极大边双连通子图)。

Whitney 定理：对任意的图 G ，有 $\kappa(G) \leq \lambda(G) \leq \delta(G)$ 。(不等式中的三项分别为点连通度、边连通度、最小度。)

稀疏图/稠密图

若一张图的边数远小于其点数的平方，那么它是一张 **稀疏图 (sparse graph)**。

若一张图的边数接近其点数的平方，那么它是一张 **稠密图 (dense graph)**。

一般用于讨论时间复杂度为 $O(|V|^2)$ 的算法与 $O(|E|)$ 的算法的效率差异。

补图/反图

补图

对于无向简单图 $G = (V, E)$ ，它的 **补图 (complement graph)** 指的是这样的一张图：记作 $\bar{G}$ ，满足 $V(\bar{G}) = V(G)$ ，且对任意节点对 (u, v) ：

$$(u, v) \notin E(G) \rightarrow (u, v) \in E(\bar{G})$$

反图

对于有向图 $G = (V, E)$ ，它的 **反图 (transpose graph)** 指的是点集不变，每条边反向得到的图，即：若 G 的反图为 $G' = (V, E')$ ，则对任意节点对 (u, v) ：

$$(u, v) \in E(G) \rightarrow (v, u) \in E(G')$$

特殊的图

完全图

完全图 (complete graph) 对于无向图 $G = (V, E)$ ，满足 $(u, v) \in E(G), 1 \leq u < v \leq |V(G)|$ ， n 阶完全图记作 K_n 。

有向完全图

有向完全图 (complete digraph)：对于有向图 $G = (V, E)$ ，满足 $(u, v) \in E(G), 1 \leq u, v \leq |V(G)|, u \neq v$ 。

无边图

无边图 (edgeless graph)：对于图 $G = (V, E)$ ，满足 $|E(G)| = 0$ ， n 阶无边图记作 $\bar{K}_n$ 或 N_n ， N_n 与 K_n 互为补图。

菊花图/星图

菊花图/星图 (star graph)：对于无向简单图 $G = (V, E)$ ，满足存在一个点 v 为支配点，其余点之间没有边相连。 $n + 1 (n > 0)$ 阶星图记作 S_n 。

链

链 (chain/path graph)：对于无向简单图 $G = (V, E)$ ，满足 $|E(G)| = |V(G)| - 1$ ，且所有边连成一条简单路径。 n 阶的链记作 P_n 。

环图

环图 (cycle graph)：对于无向简单图 $G = (V, E)$ ， $E(G)$ 恰好构成一个环。 $n (n > 2)$ 阶环图记作 C_n 。一张图为环图的充分必要条件是，它是 2-正则连通图。

轮图

轮图 (wheel graph): 对于无向简单图 $G = (V, E)$, 存在一个点 v 为支配点, 其它点之间构成一个环。 $n + 1 (n > 2)$ 阶轮图记作 W_n 。

树

树 (tree): 对于无向连通图 $G = (V, E)$, 满足 $|E(G)| = |V(G)| - 1$ 且不包含环。

基环树

基环树 (pseudotree): 对于无向连通图 $G = (V, E)$, 满足 $|E(G)| = |V(G)|$ 且恰好包含一个环。

基环外向/内向树

对于有向弱联通图 $G = (V, E)$, 如果 $d^+(v) = 1, v \in V(G)$, 则称其为一颗 **基环内向树**, 如果 $d^-(v) = 1, v \in V(G)$, 则称其为一颗 **基环外向树**。

仙人掌

仙人掌 (cactus): 对于无向连通图 $G = (V, E)$, 满足每条边最多在一个环内。

森林

多棵树可以组成一个 **森林 (forest)**, 多棵基环树可以组成 **基环森林 (pseudoforest)**, 多棵基环外向树可以组成 **基环外向树森林**, 多棵基环内向树可以组成 **基环内向森林 (functional graph)**, 多棵仙人掌可以组成 **沙漠**。

二分图

二分图 (bipartite graph): 对于图 $G = (V, E)$, V 可以被分为两部分, 每一部分的内部都没有连边。如果二分图中任何两个不在同一部分的点之间都有连边, 那么这张图是一张 **完全二分图 (complete bipartite graph/biclique)**, 一张两部分分别有 n 个点和 m 个点的完全二分图记作 $K_{n,m}$ 。

平面图

平面图 (planar graph): 对于图 G , 可以将其画在一个平面上, 且没有两条边在非端点处相交。一张图的任何子图都不是 K_5 或 $K_{3,3}$ 是其为一张平面图的充要条件。对于简单连通平面图 $G = (V, E)$ 且 $|V| \geq 3, |E| \leq 3 \times |V| - 6$ 。

竞赛图

竞赛图 (tournament graph): 对于有向简单图 G , 满足任意不同两点都恰有一条单向边。

特殊的点集/边集

独立集

对于图 $G = (V, E)$, 若 $V' \subseteq V$ 且 V' 中任意两点都不相邻, 则 V' 是图 G 的一个 **独立集 (independent set)**。

图 G 最大的独立集的大小记作 $\alpha(G)$ 。求一张图的最大独立集是 NP 困难的。

支配集

对于无向图 $G = (V, E)$, 若 $V' \subseteq V$ 且 $\forall v \in (V \setminus V')$ 存在边 $(u, v) \in E$ 满足 $u \in V'$, 则 V' 是图 G 的一个 **支配集 (dominating set)**。

无向图 G 最小的支配集的大小记作 $\gamma(G)$ 。求一张图的最小支配集是 NP 困难的。

对于有向图 $G = (E, V)$, 若 $V' \subseteq V$ 且 $\forall v \in (V \setminus V')$ 存在边 $(u, v) \in E$ 满足 $u \in V'$, 则 V' 是图 G 的一个 **出-支配集 (out-dominating set)**。类似地, 可以定义有向图的 **入-支配集 (in-dominating set)**。

有向图 G 最小的出-支配集大小记作 $\gamma^+(G)$, 最小的入-支配集大小记作 $\gamma^-(G)$ 。

边支配集

对于图 $G = (E, V)$, 若 $E' \subseteq E$ 且 $\forall e \in (E \setminus E')$ 存在 E' 中的边与其有公共点, 则称 E' 是图 G 的一个 **边支配集 (edge dominating set)**。

求一张图的最小边支配集是 NP 困难的。

匹配

对于图 $G = (V, E)$, 若 $E' \subseteq E$ 且 E' 中任意两条不同的边都没有公共的端点, 且 E' 中任意一条边都不是自环, 则 E' 是图 G 的一个 **匹配 (matching)**, 也可以叫作 **边独立集 (independent edge set)**。如果一个点是匹配中某条边的一个端点, 则称这个点是 **被匹配的 (matched)/饱和的 (saturated)**, 否则称这个点是 **不被匹配的 (unmatched)**。

边数最多的匹配被称作一张图的 **最大匹配 (maximum-cardinality matching)**。图 G 的最大匹配的大小记作 $\nu(G)$ 。

如果边带权, 那么权重之和最大的匹配被称作一张图的 **最大权匹配 (maximum-weight matching)**。

如果一个匹配在加入任何一条边后都不再是一个匹配, 那么这个匹配是一个 **极大匹配 (maximal matching)**。最大的极大匹配就是最大匹配, 任何最大匹配都是极大匹配。极大匹配一定是边支配集, 但边支配集不一定是匹配。最小极大匹配和最小边支配集大小相等, 但最小边支配集不一定是匹配。求最小极大匹配是 NP 困难的。

如果在一个匹配中所有点都是被匹配的, 那么这个匹配是一个 **完美匹配 (perfect matching)**。如果在一个匹配中只有一个点不被匹配, 那么这个匹配是一个 **准完美匹配 (near-perfect matching)**。

对于一个匹配 M , 若一条路径以非匹配点为起点, 每相邻两条边的其中一条在匹配中而另一条不在匹配中, 则这条路径被称作一条 **交替路径 (alternating path)**; 一条在非匹配点终止的交替路径, 被称作一条 **增广路径 (augmenting path)**。

托特定理: n 阶无向图 G 有完美匹配当且仅当对于任意的 $V' \subset V(G)$, $p_{\text{odd}}(G \setminus V') \leq |V'|$, 其中 p_{odd} 表示奇数阶连通分支数。

托特定理 (推论): 任何无桥 3-正则图都有完美匹配。

点覆盖

对于图 $G = (V, E)$, 若 $V' \subseteq V$ 且 $\forall e \in E$ 满足 e 的至少一个端点在 V' 中, 则称 V' 是图 G 的一个 **点覆盖 (vertex cover)**。

点覆盖集必为支配集, 但极小点覆盖集不一定是极小支配集。

一个点集是点覆盖的充要条件是其补集是独立集, 因此最小点覆盖的补集是最大独立集。求一张图的最小点覆盖是 NP 困难的。

一张图的任何一个匹配的大小都不超过其任何一个点覆盖的大小。完全二分图 $K_{n,m}$ 的最大匹配和最小点覆盖大小都为 $\min(n, m)$ 。

边覆盖

对于图 $G = (V, E)$ ，若 $E' \subseteq E$ 且 $\forall v \in V$ 满足 v 与 E' 中的至少一条边相邻，则称 E' 是图 G 的一个 **边覆盖 (edge cover)**。

最小边覆盖的大小记作 $\rho(G)$ ，可以由最大匹配贪心扩展求得：对于所有非匹配点，将其一条邻边加入最大匹配中，即得到了一个最小边覆盖。

最大匹配也可以由最小边覆盖求得：对于最小边覆盖中每对有公共点的边删去其中一条。

一张图的最小边覆盖的大小加上最大匹配的大小等于图的点数，即 $\rho(G) + \nu(G) = |V(G)|$ 。

一张图的最大匹配的大小不超过最小边覆盖的大小，即 $\nu(G) \leq \rho(G)$ 。特别地，完美匹配一定是一个最小边覆盖，这也是上式取到等号的唯一情况。

一张图的任何一个独立集的大小都不超过其任何一个边覆盖的大小。完全二分图 $K_{n,m}$ 的最大独立集和最小边覆盖大小都为 $\max(n, m)$ 。

团

对于图 $G = (V, E)$ ，若 $V' \subseteq V$ 且 V' 中任意两个不同的顶点都相邻，则 V' 是图 G 的一个 **团 (clique)**。团的导出子图是完全图。

如果一个团在加入任何一个顶点后都不再是一个团，则这个团是一个 **极大团 (maximal clique)**。

一张图的最大团的大小记作 $\omega(G)$ ，最大团的大小等于其补图最大独立集的大小，即 $\omega(G) = \alpha(\bar{G})$ 。求一张图的最大团是 NP 困难的。

图的存储

直接存储

实现

```
1  const int M=2e5+5;
2  struct node
3  {
4      int u,v,w;
5  }edge[M];
6  int main()
7  {
8      int n,m,cnt=0;
9      cin>>n>>m;
10     for(int i=0;i<m;++i)
11     {
12         int u,v,w;
13         cin>>u>>v>>w;
14         edge[cnt++]={u,v,w};
15         edge[cnt++]={v,u,w}; //无向边
16     }
17 }
```

复杂度

查询是否存在某条边： $O(m)$ 。

遍历一个点的所有出边： $O(m)$ 。

遍历整张图： $O(nm)$ 。

空间复杂度： $O(m)$ 。

应用

效率较低，一般不使用，在 **Kruskal算法** 中，由于需要把边按权排序，所以需要直接存边。

邻接矩阵

实现

```
1  const int N=1e3+5;
2  int adj[N][N];
3  int main()
4  {
5      int n,m;
6      memset(adj,-1,sizeof(adj));
7      for(int i=0;i<m;++i)
8      {
9          int u,v,w;
10         cin>>u>>v>>w;
11         adj[u][v]=w;
12         adj[v][u]=w; //无向边
13     }
14 }
```

复杂度

查询是否存在某条边： $O(1)$ 。

遍历一个点的所有出边： $O(n)$ 。

遍历整张图： $O(n^2)$ 。

空间复杂度： $O(n^2)$ 。

应用

邻接矩阵只适用于没有重边（或重边可以忽略）的情况。

其最显著的优点是可以 $O(1)$ 查询一条边是否存在。

由于邻接矩阵在稀疏图上效率很低（尤其是在点数较多的图上，空间无法承受），所以一般只会在稠密图上使用邻接矩阵。

邻接表

实现

```

1  const int N=2e5+5;
2  vector<pair<int,int>>vec[N];
3  int main()
4  {
5      int n,m;
6      for(int i=0;i<m;++i)
7      {
8          int u,v,w;
9          cin>>u>>v>>w;
10         vec[u].push_back({v,w});
11         vec[v].push_back({u,w}); //无向边
12     }
13 }

```

复杂度

查询是否存在 u 到 v 的边: $O(d^+(u))$ 。

遍历点 u 的所有出边: $O(d^+(u))$ 。

遍历整张图: $O(n + m)$ 。

空间复杂度: $O(m)$ 。

应用

存各种图都很适合，除非有特殊需求（如需要快速查询一条边是否存在，且点数较少，可以使用邻接矩阵）。

尤其适用于需要对一个点的所有出边进行排序的场合。

链式前向星

实现

```

1  const int M=2e5+5;
2  int head[N],cnt;
3  struct node
4  {
5      int v,w,nxt;
6  }edge[M];
7  void add(int u,int v,int w)
8  {
9      edge[++cnt].nxt=head[u];
10     edge[cnt].v=v;
11     edge[cnt].w=w;
12     head[u]=cnt;
13 }
14 int main()
15 {
16     int n,m;
17     for(int i=0;i<m;++i)
18     {
19         int u,v,w;
20         cin>>u>>v>>w;
21         add(u,v,w);
22         add(v,u,w); //无向边
23     }

```

复杂度

查询是否存在 u 到 v 的边： $O(d^+(u))$ 。

遍历点 u 的所有出边： $O(d^+(u))$ 。

遍历整张图： $O(n + m)$ 。

空间复杂度： $O(m)$ 。

应用

存各种图都很适合，但不能快速查询一条边是否存在，也不能方便地对一个点的出边进行排序。

优点是边是带编号的，有时会非常有用，而且如果 `cnt` 的初始值为奇数，存双向边时 $i \wedge 1$ 即是 i 的反边（常用于网络流）。

DFS

过程

递归调用搜索，每次都尝试向**深度更深**的节点访问，对已经访问过的点打上标记以**不再次访问**。

一般结构的伪代码如下：

```

1  DFS(v) // v 可以是图中的一个顶点，也可以是抽象的概念，如 dp 状态等。
2      在 v 上打访问标记
3      for u in v 的相邻节点
4          if u 没有打过访问标记 then
5              DFS(u)
6          end
7      end
8  end

```

通常时间复杂度为 $O(n + m)$ ，空间复杂度为 $O(n)$ 。

无环图

邻接矩阵

```

1  const int N=1e3+5;
2  int adj[N][N];
3  void dfs(int u,int pre)
4  {
5      for(int i=1;i<=n;++i)
6      {
7          if(adj[u][i]==-1||i==pre)continue;
8          dfs(i,u);
9      }
10 }
11 int main()
12 {
13     int n,m;
14     memset(adj,-1,sizeof(adj));
15     for(int i=0;i<m;++i)

```

```

16     {
17         int u,v,w;
18         cin>>u>>v>>w;
19         adj[u][v]=w;
20         adj[v][u]=w;//无向边
21     }
22     dfs(1,0);
23 }

```

邻接表

```

1  const int N=2e5+5;
2  vector<pair<int,int>>vec[N];
3  void dfs(int u,int pre)
4  {
5      for(auto [v,w]:vec[u])
6      {
7          if(v==pre)continue;
8          dfs(v,u);
9      }
10 }
11 int main()
12 {
13     int n,m;
14     for(int i=0;i<m;++i)
15     {
16         int u,v,w;
17         cin>>u>>v>>w;
18         vec[u].push_back({v,w});
19         vec[v].push_back({u,w}); //无向边
20     }
21     dfs(1,0);
22 }

```

链式前向星

```

1  const int M=2e5+5;
2  int head[N],cnt;
3  struct node
4  {
5      int v,w,nxt;
6  }edge[M];
7  void add(int u,int v,int w)
8  {
9      edge[++cnt].nxt=head[u];
10     edge[cnt].v=v;
11     edge[cnt].w=w;
12     head[u]=cnt;
13 }
14 void dfs(int u,int pre)
15 {
16     for(int i=head[u];i;i=edge[i].nxt)
17     {
18         int v=edge[i].v,w=edge[i].w;
19         if(v==pre)continue;
20         dfs(v,u);

```

```

21     }
22 }
23 int main()
24 {
25     int n,m;
26     for(int i=0;i<m;++i)
27     {
28         int u,v,w;
29         cin>>u>>v>>w;
30         add(u,v,w);
31         add(v,u,w);//无向边
32     }
33     dfs(1,0);
34 }

```

DFS序

DFS 序即 DFS 调用过程中访问的节点编号的序列。

其中每个子树都对应 DFS 序列中的一个连续段。

括号序列

DFS 进入某个节点的时候记录一个左括号 (，退出某个节点的时候记录一个右括号)。

每个节点会出现两次。相邻两个节点的深度相差 0/1。

一般图

对于非连通图，只能访问到起点所在的连通分量。

对于连通图，DFS 序通常不唯一。

在 DFS 过程中，通过记录每个节点从哪个点访问而来，可以建立一个树结构，称为 DFS 树。DFS 树是原图的一个生成树。

BFS

过程

每次都尝试访问 **同一层** 的节点，访问完当前层的所有节点再访问下一层的节点。

这样做的结果是，BFS 算法找到的路径是从起点开始的 **最短** 合法路径。换言之，这条路径所包含的边数最小。

在 BFS 结束时，每个节点都是通过从起点到该点的最短路径访问的。

具体说，一般使用一个队列 Q 来记录要处理的节点，然后开一个布尔数组 vis[] 来标记是否已经访问过某个节点。

开始的时候，我们将所有节点的 vis 值设为 0，表示没有访问过；然后把起点 s 放入队列 Q 中并将 vis[s] 设为 1。

之后，我们每次从队列 Q 中取出队首的节点 u，然后把与 u 相邻的所有节点 v 标记为已访问过并放入队列 Q。

循环直至当队列 Q 为空，表示 BFS 结束。

一般结构的伪代码如下：

```

1  bfs(s)
2  {
3      q = new queue()
4      q.push(s), visited[s] = true
5      while (!q.empty())
6      {
7          u = q.pop()
8          for each edge(u, v)
9          {
10             if (!visited[v])
11             {
12                 q.push(v)
13                 visited[v] = true
14             }
15         }
16     }
17 }
18

```

通常时间复杂度为 $O(n + m)$ ，空间复杂度为 $O(n)$ 。

无环图

```

1  const int N=2e5+5;
2  vector<int>vec[N];
3  int fa[N];
4  void bfs(int s)
5  {
6      vector<int>dis(n+1,1e9),vis(n+1);
7      queue<int>q;
8      q.push(s);
9      vis[s]=1;
10     dis[s]=0;
11     fa[s]=-1;
12     while(!q.empty())
13     {
14         int u=q.front();
15         q.pop();
16         for(auto v:vec[u])if(!vis[v])
17         {
18             q.push(v);
19             vis[v]=1;
20             dis[v]=dis[u]+1;
21             fa[v]=u;
22         }
23     }
24 }
25 void restore(int x)
26 {
27     vector<int>v;
28     for(int i=x;i!=-1;i=fa[i])v.push_back(i);
29     reverse(v.begin(),v.end());
30     for(auto x:v)cout<<x<<" ";
31     cout<<"\n";
32 }

```

在 BFS 的过程中，也可以记录一些额外的信息。例如上述代码中，`dis` 数组用于记录起点到某个节点的最短距离（要经过的最少边数），`fa` 数组记录是从哪个节点走到当前节点的。

有了 `dis` 数组，可以方便地得到起点到一个节点的距离。

有了 `fa` 数组，可以方便地还原出起点到一个点的最短路径。上述代码中的 `restore()` 函数使用该数组依次输出从起点到节点 `x` 的最短路径所经过的节点。

BFS序

BFS 序即 BFS 调用过程中访问的节点编号的序列。

一般图

如果原图不连通，只能访问到从起点出发能够到达的点。

BFS 序列通常也不唯一。

类似的我们也可以定义 BFS 树：在 BFS 过程中，通过记录每个节点从哪个点访问而来，可以建立一个树结构，即为 BFS 树。

01BFS

适用于边权值为 0/1 的最短路问题。

我们把没有权值的边扩展到的点放到队首，有权值的边扩展到的点放到队尾。这样即可保证像普通 BFS 一样整个队列队首到队尾权值单调不下降。

一般结构的伪代码如下：

```
1  while (队列不为空)
2  {
3      int u = 队首;
4      弹出队首;
5      for (枚举 u 的邻居)
6      {
7          更新数据
8          if (...)
9              添加到队首;
10         else
11             添加到队尾;
12     }
13 }
```

最短路

性质

对于边权为**正**的图，任意两个结点之间的最短路，不会经过重复的结点。

对于边权为**正**的图，任意两个结点之间的最短路，不会经过重复的边。

对于边权为**正**的图，任意两个结点之间的最短路，结点数不会超过 n ，边数不会超过 $n - 1$ 。

记号

- n 为图上点的数目, m 为图上边的数目;
- s 为最短路的源点;
- $D(u)$ 为点 s 到点 u 的 **实际** 最短路长度;
- $dis(u)$ 为点 s 到点 u **最短路算法转移时**的最短路长度。 $dis(u) \geq D(u)$ 。特别地, 当最短路算法结束时, $dis(u) = D(u)$ 。
- $w(u, v)$ 为 (u, v) 这一条边的边权。

Floyd

可用于求解任意两点间的最短路, 仅要求图中 **不能存在负环 (最短路存在)**。

定义数组 $f[k][x][y]$, 表示只允许经过结点 1 到 k , 结点 x 到结点 y 的最短路长度。

显然, $f[n][x][y]$ 就是结点 x 到结点 y 的最短路长度。

考虑如何求出 f 数组的值。

$f[0][x][y]$: x 与 y 的边权, 或者 0 , 或者 $+\infty$ 。

$f[k][x][y] = \min(f[k-1][x][y], f[k-1][x][k] + f[k-1][k][y])$ ($f[k-1][x][y]$, 为不经过 k 点的最短路径, 而 $f[k-1][x][k] + f[k-1][k][y]$, 为经过了 k 点的最短路)。

注意到 $f[k][x][y]$ 的涵义是第一维为 $k-1$ 这一行和这一列的所有元素的最小值, 包含了 $f[k-1][x][y]$, 那么直接进行更改也不会改变最小值的值。因为如果将该三维矩阵压缩为二维, 则所求结果 $f[x][y]$ 一开始即为原 $f[k-1][x][y]$ 的值, 最后依然会成为该行和该列的最小值。故我们可以压缩第一维的空间。

实现如下:

```
1 for(k=1;k<=n;++k)
2     for(x=1;x<=n;++x)
3         for(y=1;y<=n;++y)
4             f[x][y]=min(f[x][y], f[x][k]+f[k][y]);
```

时间复杂度为 $O(n^3)$, 空间复杂度为 $O(n^2)$ 。

Bellman-Ford

对于边, 我们使用 $dis(v) = \min(dis(v), dis(u) + w(u, v))$ 来不断更新 $dis(v)$, 我们称其为对于 (u, v) 的松弛操作。

Bellman-Ford 算法就是从出发点不断尝试对每一条边尝试松弛, 当一次对于所有边的尝试都失败时停止操作。

考虑到最短路边数最多为 $n - 1$ 条, 每轮尝试松弛 m 条边, 故总复杂度为 $O(nm)$ 。

考虑到最多进行 $n - 1$ 轮松弛, 故如果 Bellman-Ford 算法的松弛轮数为 n 时, 代表从 S 出发能到达一个负环 (最短路不存在)。

注意: 通过松弛轮数判断负环只能确保 S 出发能否到达负环, 实际实现时应建立一个超级源点 S' , 与每一个节点连一条权值为 0 的边, 再以 S' 为起点执行 Bellman-Ford 算法。

实现如下:

```
1 const int N=1e3+5;
2 vector<pair<int,int>>vec[N];
```

```

3  bool BellmanFord(int s)
4  {
5      vector<int>dis(n+1,1e9);
6      dis[s]=0;
7      bool f=1;//判断一轮循环过程中是否发生松弛操作
8      for(int i=1;i<=n;++i)
9      {
10         f=0;
11         for(int u=1;u<=n;++u)
12         {
13             if(dis[u]==1e9)continue;
14             for(auto [v,w]:vec[u])
15             {
16                 if(dis[v]>dis[u]+w)
17                 {
18                     dis[v]=dis[u]+w;
19                     f=1;
20                 }
21             }
22         }
23         if(!f)break;//没有可以松弛的边时就停止
24     }
25     return f;//第n轮循环仍可以松弛时说明s可以抵达一个负环
26 }

```

Dijkstra

要求求解最短路的图为 **非负权图**。

考虑将结点分成两个集合：已确定最短路长度的点集，记为 S ；未确定最短路长度的点集，记为 T 。一开始所有的点都属于 T 。

初始化 $dis(s) = 0$ ，其他点的 dis 均为 $+\infty$ 。

然后重复以下操作：

1. 从 T 集合中，选取一个最短路长度最小的结点，移到 S 集合中。
2. 对那些刚刚被加入 S 集合的结点的所有出边执行松弛操作。

T 集合为空，算法结束。

有多种方法来维护 1 操作中最短路长度最小的结点，不同的实现导致了 Dijkstra 算法时间复杂度上的差异。

- 暴力：不使用任何数据结构进行维护，每次 2 操作执行完毕后，直接在 T 中暴力寻找最短路长度最小的结点。2 操作总时间复杂度为 $O(m)$ ，1 操作总时间复杂度为 $O(n^2)$ ，全过程的时间复杂度为 $O(n^2)$ 。
- 二叉堆：每成功松弛一条边 (u, v) ，就将 v 插入二叉堆中（如果 v 已经在二叉堆中，直接修改相应元素的权值），1 操作直接取堆顶结点即可。共计 $O(m)$ 次二叉堆上的插入（修改）操作， $O(n)$ 次删除堆顶操作，而插入（修改）和删除的时间复杂度均为 $O(\log n)$ ，时间复杂度为 $O(m \log n)$ 。
- 优先队列：和二叉堆类似，但使用优先队列时，如果同一个点的最短路被更新多次，因为先前更新时插入的元素不能被删除，也不能被修改，只能留在优先队列中，故优先队列内的元素个数是 $O(m)$ 的，时间复杂度为 $O(m \log m)$ 。
- Fibonacci 堆：和前面二者类似，但 Fibonacci 堆插入的时间复杂度为 $O(1)$ ，故时间复杂度为 $O(n \log n + m)$ ，时间复杂度最优。但 Fibonacci 堆较二叉堆不易实现，效率优势也不够大，算法竞赛中较少使用。

- 线段树：和二叉堆原理类似，不过将每次成功松弛后插入二叉堆的操作改为在线段树上执行单点修改，而 1 操作则是线段树上的全局查询最小值。时间复杂度为 $O(m \log n)$ 。

一般最常用的为优先队列实现，给出一个实现：

```
1 namespace dijkstra
2 {
3     #define ll long long
4     struct edge
5     {
6         ll v,w;
7     };
8     struct node
9     {
10        ll u,w;
11        bool operator >(const node &a)const{
12            return w>a.w;
13        }
14    };
15    vector<ll>dis;
16    vector<edge>vec[N];
17    void clear(ll n)
18    {
19        for(int i=1;i<=n;++i)vec[i].clear();
20        return ;
21    }
22    void add(ll u,edge x)
23    {
24        vec[u].push_back(x);
25        return ;
26    }
27    void dij(ll S,ll n)
28    {
29        priority_queue<node,vector<node>,greater<node>>q;
30        vector<ll>di(n+1,1e18),vis(n+1,0);
31        di[S]=0;
32        q.push({S,0});
33        while(!q.empty())
34        {
35            ll u=q.top().u;
36            q.pop();
37            if(vis[u])continue;
38            vis[u]=1;
39            for(auto x:vec[u])
40            {
41                ll v=x.v,w=x.w;
42                if(di[v]>di[u]+w)
43                {
44                    di[v]=di[u]+w;
45                    q.push({v,di[v]});
46                }
47            }
48        }
49        dis=di;
50    }
51 }
```

例题

P1491 集合位置

题意：求解非严格次短路

思路：找出一条最短路，之后一次去掉一条最短路上的边，对于每一次去掉一条边后的图都跑一次最短路，其中最小的答案就是次短路。

最小生成树

定义

定义无向连通图的 **最小生成树**（Minimum Spanning Tree, MST）为边权和最小的生成树。

注意：只有连通图才有生成树，而对于非连通图，只存在生成森林。

Kruskal

该算法的基本思想是贪心地从小到大加入边。

具体实现为维护一个森林，查询两个结点是否在同一棵树中，连接两棵树。

其中，查询两点是否连通和连接两点可以使用并查集维护。

考虑使用按秩合并或路径压缩的并查集维护，总时间复杂度为 $O(m \log m)$ 。

实现如下：

```
1  #define ll long long
2  struct node
3  {
4      ll u,v,w;
5      bool operator<(const node& i)const{
6          return w>i.w;}
7  }edge[N];
8  void init()
9  {
10     for(int i=1;i<=n;++i)pre[i]=i;
11     return ;
12 }
13 ll find(ll x)
14 {
15     if(x!=pre[x])pre[x]=find(pre[x]);
16     return pre[x];
17 }
18 bool unite(ll x,ll y)
19 {
20     if(find(x)!=find(y))
21     {
22         pre[find(x)]=find(y);
23         return true;
24     }
25     return false;
26 }
27 void kruskal()
28 {
```

```

29     init();
30     sort(edge, edge+m);
31     ans=0;
32     for(int i=0; i<m; ++i) if(unite(edge[i].u, edge[i].v)) ans+=edge[i].w;
33 }

```

Prim

该算法的基本思想是从一个结点开始，不断选择距离最小的一个结点加边，然后用新的边更新其他结点的距离。

与 Dijkstra 算法相似，具体时间复杂度取决于维护距离最小点的方式。

暴力： $O(n^2 + m)$ 。

二叉堆： $O((n + m) \log n)$ 。

Fib 堆： $O(n \log n + m)$ 。

实现如下：

```

1  struct node
2  {
3      int first, second;
4      bool operator < (const node &x) const {
5          return first > x.first;
6      }
7  };
8  void prim()
9  {
10     ans=0;
11     priority_queue<node> q;
12     vector<int> dis(n+1, 1e9), vis(n+1);
13     dis[1]=0;
14     q.push({0, 1});
15     int cnt=0;
16     while(!q.empty() && cnt<n-1)
17     {
18         int u=q.top().second;
19         q.pop();
20         if(!vis[u])
21         {
22             vis[u]=1;
23             cnt++;
24             ans+=dis[u];
25             for(auto [v, w]: vec[u])
26             {
27                 if(dis[v]>w)
28                 {
29                     dis[v]=w;
30                     q.push({dis[v], v});
31                 }
32             }
33         }
34     }
35 }

```

拓扑排序

定义

拓扑排序要解决的问题是给一个图的所有节点排序。

在一个 DAG（有向无环图）中，我们将图中的顶点以线性方式进行排序，使得对于任何的顶点 u 到 v 的有向边 (u, v) ，都可以有 u 在 v 的前面。

或者说对于一个 DAG，如果从 u 到 v 有边，则认为 v 依赖于 u 。如果 u 到 v 有路径，则称 u 间接依赖于 v 。

拓扑排序的目标是将所有节点排序，使得排在前面的节点不能依赖于排在后面的节点。

Kahn

考虑维护一个入度为 0 的顶点的集合 S 。

初始状态下，集合 S 装着所有入度为 0 的点， L 是一个空列表。

每次从 S 中取出一个点 u 放入 L ，然后将 u 的所有邻边删除。对于边 (u, v) ，将该边删除后点 v 的入度减一，若减一后 v 的入度为 0，则将 v 放入 S 中。

不断重复以上过程，直到集合 S 为空。检查图中是否存在任何边，如果有，那么这个图一定有环路，否则返回 L ， L 中顶点的顺序就是拓扑排序的结果。

实现如下：

```
1  vector<ll>vec[N],L;
2  ll ind[N];
3  bool toop(ll s)
4  {
5      queue<ll>q;
6      for(int i=1;i<=n;++i)if(ind[i]==0)q.push(i);
7      while(!q.empty())
8      {
9          ll u=q.front();
10         q.pop();
11         L.push_back(u);
12         for(auto v:vec[u])
13         {
14             ind[v]--;
15             if(ind[v]==0)q.push(v);
16         }
17     }
18     if(L.size()!=n)return 0;
19     return 1;
20 }
```

考虑初始化和集合操作需要对图遍历，故总复杂度为 $O(n + m)$ 。

应用

除去直接求得 DAG 的拓扑序外，还可以用来判断图中是否有环，判断图是否是一条链。

求字典序最大/最小的拓扑序

将 Kahn 算法中的队列替换成最大堆/最小堆实现的优先队列即可，此时时间复杂度为 $O(m + n \log n)$ 。

二分图

定义

存在一种划分，使得节点由两个集合组成，且两个集合内部没有边的图。

如果两个集合中的点分别染成黑色和白色，可以发现二分图中的每一条边都一定是连接一个黑色点和一个白色点，我们可以利用这个性质进行二分图判定（DFS 染色随后查询每条边）。

考虑每一条边都是从一个集合走到另一个集合，故只有走偶数次才可能回到同一个集合，所以二分图不存在长度为奇数的环。

二分图最大匹配

增广路算法

因为增广路长度为奇数，路径起始点非左即右，所以我们先考虑从左边的未匹配点找增广路。注意到因为交错路的关系，增广路上的第奇数条边都是非匹配边，第偶数条边都是匹配边，于是左到右都是非匹配边，右到左都是匹配边。于是我们给二分图 **定向**，问题转换成，有向图中从给定起点找一条简单路径走到某个未匹配点，此问题等价给定起始点 s 能否走到终点 t 。那么只要从起始点开始 DFS 遍历直到找到某个未匹配点。未找到增广路时，我们拓展的路也称为 **交错树**。

考虑到要枚举 n 个点，每次遍历图，故总复杂度为 $O(nm)$ 。

实现：

```
1 struct augment_path {
2     vector<vector<int>> > g;
3     vector<int> pa; // 匹配
4     vector<int> pb;
5     vector<int> vis; // 访问
6     int n, m; // 两个点集中的顶点数量
7     int dfn; // 时间戳记
8     int res; // 匹配数
9
10    augment_path(int _n, int _m) : n(_n), m(_m) {
11        assert(0 <= n && 0 <= m);
12        pa = vector<int>(n, -1);
13        pb = vector<int>(m, -1);
14        vis = vector<int>(n);
15        g.resize(n);
16        res = 0;
17        dfn = 0;
18    }
19
20    void add(int from, int to) {
21        assert(0 <= from && from < n && 0 <= to && to < m);
22        g[from].push_back(to);
23    }
24
25    bool dfs(int v) {
26        vis[v] = dfn;
```

```

27     for (int u : g[v]) {
28         if (pb[u] == -1) {
29             pb[u] = v;
30             pa[v] = u;
31             return true;
32         }
33     }
34     for (int u : g[v]) {
35         if (vis[pb[u]] != dfn && dfs(pb[u])) {
36             pa[v] = u;
37             pb[u] = v;
38             return true;
39         }
40     }
41     return false;
42 }
43
44 int solve() {
45     while (true) {
46         dfn++;
47         int cnt = 0;
48         for (int i = 0; i < n; i++) {
49             if (pa[i] == -1 && dfs(i)) {
50                 cnt++;
51             }
52         }
53         if (cnt == 0) {
54             break;
55         }
56         res += cnt;
57     }
58     return res;
59 }
60 };

```

二分图最大权匹配

KM算法

二分图的最大权匹配是指二分图中边权和最大的匹配。

匈牙利算法又称为 **KM 算法**，可以在 $O(n^3)$ 时间内求出二分图的 **最大权完美匹配**。

考虑到二分图中两个集合中的点并不总是相同，为了能应用 KM 算法解决二分图的最大权匹配，需要先作如下处理：将两个集合中点数比较少的补点，使得两边点数相同，再将不存在的边权重设为 0，这种情况下，问题就转换成求 **最大权完美匹配问题**，从而能应用 KM 算法求解。

可行顶标：给每个节点 i 分配一个权值 $l(i)$ ，对于所有边 (u, v) 满足 $w(u, v) \leq l(u) + l(v)$ 。

相等子图：在一组可行顶标下原图的生成子图，包含所有点但只包含满足 $w(u, v) = l(u) + l(v)$ 的边 (u, v) 。

定理 1：对于某组可行顶标，如果其相等子图存在完美匹配，那么，该匹配就是原二分图的最大权完美匹配。

证明 1：

考虑原二分图任意一组完美匹配 M ，其边权和为

$$val(M) = \sum_{(u,v) \in M} w(u,v) \leq \sum_{(u,v) \in M} l(u) + l(v) \leq \sum_{i=1}^n l(i)$$

任意一组可行顶标的相等子图的完美匹配 M' 的边权和

$$val(M') = \sum_{(u,v) \in M} l(u) + l(v) = \sum_{i=1}^n l(i)$$

即任意一组完美匹配的边权和都不会大于 $val(M')$ ，那么 M' 就是最大权匹配。

有了定理 1，我们的目标就是透过不断的调整可行顶标，使得相等子图是完美匹配。

因为两边点数相等，假设点数为 n ， $lx(i)$ 表示左边第 i 个点的顶标， $ly(i)$ 表示右边第 i 个点的顶标， $w(u,v)$ 表示左边第 u 个点和右边第 v 个点之间的权重。

首先初始化一组可行顶标，例如 $lx(i) = \max_{1 \leq j \leq n} \{w(i,j)\}$, $ly(i) = 0$

然后选一个未匹配点，如同最大匹配一样求增广路。找到增广路就增广，否则，会得到一个交错树。

令 S, T 表示二分图左边右边在交错树中的点， S', T' 表示不在交错树中的点。

在相等子图中：

- $S - T'$ 的边不存在，否则交错树会增长。
- $S' - T$ 一定是非匹配边，否则他就属于 S 。

假设给 S 中的顶标 $-a$ ，给 T 中的顶标 $+a$ ，可以发现

- $S - T$ 边依然存在相等子图中。
- $S' - T'$ 没变化。
- $S - T'$ 中的 $lx + ly$ 会减少，有可能加入相等子图。
- $S' - T$ 中的 $lx + ly$ 会增加，不可能加入相等子图。

故 a 应当为 $S - T'$ 当中最小的边权，也即

$$a = \min\{lx(u) + ly(v) - w(u,v) | u \in S, v \in T'\}$$

当一条新的边 (u,v) 加入相等子图后有两种情况

- v 是未匹配点，则找到增广路
- v 和 S' 中的点已经匹配

这样至多修改 n 次顶标后，就可以找到增广路。

每次修改顶标的时候，交错树中的边不会离开相等子图，那么我们直接维护这棵树。

我们对 T 中的每个点 v 维护

$$slack(v) = \min\{lx(u) + ly(v) - w(u,v) | u \in S\}$$

所以可以在 $O(n)$ 算出顶标修改值 $a = \min\{slack(v) | v \in T'\}$

交错树新增一个点进入 S 的时候需要 $O(n)$ 更新 $slack(v)$ 。修改顶标需要 $O(n)$ 给每个 $slack(v)$ 减去 a 。只要交错树找到一个未匹配点，就找到增广路。

一开始枚举 n 个点找增广路，为了找增广路需要延伸 n 次交错树，每次延伸需要 n 次维护，总复杂度 $O(n^3)$ 。

实现：

```
1 template <typename T>
2 struct hungarian { // km
```

```

3   int n;
4   vector<int> matchx; // 左集合对应的匹配点
5   vector<int> matchy; // 右集合对应的匹配点
6   vector<int> pre;    // 连接右集合的左点
7   vector<bool> visx;  // 拜访数组 左
8   vector<bool> visy;  // 拜访数组 右
9   vector<T> lx;
10  vector<T> ly;
11  vector<vector<T> > g;
12  vector<T> slack;
13  T inf;
14  T res;
15  queue<int> q;
16  int org_n;
17  int org_m;
18
19  hungarian(int _n, int _m) {
20      org_n = _n;
21      org_m = _m;
22      n = max(_n, _m);
23      inf = numeric_limits<T>::max();
24      res = 0;
25      g = vector<vector<T> >(n, vector<T>(n));
26      matchx = vector<int>(n, -1);
27      matchy = vector<int>(n, -1);
28      pre = vector<int>(n);
29      visx = vector<bool>(n);
30      visy = vector<bool>(n);
31      lx = vector<T>(n, -inf);
32      ly = vector<T>(n);
33      slack = vector<T>(n);
34  }
35
36  void addEdge(int u, int v, int w) {
37      g[u][v] = max(w, 0); // 负值还不如不匹配 因此设为0不影响
38  }
39
40  bool check(int v) {
41      visy[v] = true;
42      if (matchy[v] != -1) {
43          q.push(matchy[v]);
44          visx[matchy[v]] = true; // in S
45          return false;
46      }
47      // 找到新的未匹配点 更新匹配点 pre 数组记录着"非匹配边"上与之相连的点
48      while (v != -1) {
49          matchy[v] = pre[v];
50          swap(v, matchx[pre[v]]);
51      }
52      return true;
53  }
54
55  void bfs(int i) {
56      while (!q.empty()) {
57          q.pop();
58      }
59      q.push(i);
60      visx[i] = true;

```

```

61     while (true) {
62         while (!q.empty()) {
63             int u = q.front();
64             q.pop();
65             for (int v = 0; v < n; v++) {
66                 if (!visy[v]) {
67                     T delta = lx[u] + ly[v] - g[u][v];
68                     if (slack[v] >= delta) {
69                         pre[v] = u;
70                         if (delta) {
71                             slack[v] = delta;
72                         } else if (check(v)) { // delta=0 代表有机会加入相等子图 找增广路
73                                                 // 找到就return 重建交错树
74                             return;
75                         }
76                     }
77                 }
78             }
79         }
80         // 没有增广路 修改顶标
81         T a = inf;
82         for (int j = 0; j < n; j++) {
83             if (!visy[j]) {
84                 a = min(a, slack[j]);
85             }
86         }
87         for (int j = 0; j < n; j++) {
88             if (visx[j]) { // S
89                 lx[j] -= a;
90             }
91             if (visy[j]) { // T
92                 ly[j] += a;
93             } else { // T'
94                 slack[j] -= a;
95             }
96         }
97         for (int j = 0; j < n; j++) {
98             if (!visy[j] && slack[j] == 0 && check(j)) {
99                 return;
100             }
101         }
102     }
103 }
104
105 void solve() {
106     // 初始顶标
107     for (int i = 0; i < n; i++) {
108         for (int j = 0; j < n; j++) {
109             lx[i] = max(lx[i], g[i][j]);
110         }
111     }
112
113     for (int i = 0; i < n; i++) {
114         fill(slack.begin(), slack.end(), inf);
115         fill(visx.begin(), visx.end(), false);
116         fill(visy.begin(), visy.end(), false);
117         bfs(i);
118     }

```

```

119
120     // custom
121     for (int i = 0; i < n; i++) {
122         if (g[i][matchx[i]] > 0) {
123             res += g[i][matchx[i]];
124         } else {
125             matchx[i] = -1;
126         }
127     }
128     cout << res << "\n";
129     for (int i = 0; i < org_n; i++) {
130         cout << matchx[i] + 1 << " ";
131     }
132     cout << "\n";
133 }
134 };

```

欧拉图

定义

- **欧拉回路**：通过图中每条边恰好一次的回路
- **欧拉通路**：通过图中每条边恰好一次的通路
- **欧拉图**：具有欧拉回路的图
- **半欧拉图**：具有欧拉通路但不具有欧拉回路的图

性质

欧拉图中所有顶点的度数都是偶数。

欧拉图是若干个环的并，且每条边被包含在奇数个环内。

1. 无向图是欧拉图当且仅当：
 - 非零度顶点是连通的
 - 顶点的度数都是偶数
2. 无向图是半欧拉图当且仅当：
 - 非零度顶点是连通的
 - 恰有 0 或 2 个奇度顶点
3. 有向图是欧拉图当且仅当：
 - 非零度顶点是强连通的
 - 每个顶点的入度和出度相等
4. 有向图是半欧拉图当且仅当：
 - 非零度顶点是弱连通的
 - 至多一个顶点的出度与入度之差为 1
 - 至多一个顶点的入度与出度之差为 1
 - 其他顶点的入度和出度相等

求欧拉回路或欧拉路

CF1634E Fair Share

题意：给出 m 个偶数长度的数字串，要把全部数字分到两个集合 L, R 中，使得每个串中一半数字在 L ，另一半在 R ，且最终两个集合相同。若可以构造，给出一种构造；若无法构造，则输出 **NO**。

```
1 3
2 2
3 1 2
4 4
5 1 2 3 3
6 6
7 1 1 2 2 3 3
```

思路：如果有的值出现了奇数次，那显然是不行的。否则，构造一种分 L, R 的方案来说明一定可行。

将数组和每种数值都看作图中的点，每次数值 x 在第 y 个数组出现，在 x, y 分别对应的点之间连一条无向边（边上储存位置，以便最终答案的输出）。这样构成的无向图（不一定简单，不一定连通），每个点的度都是偶数，所以每个连通分量必然存在一条欧拉回路。

将每条边按欧拉回路的方向定向。

这样，每个代表数值的点和代表数组的点的出入度相等了。我们只需要将 数组 $\rightarrow$ 数值 的那些位置放入 L ，将 数值 $\rightarrow$ 数组 的那些位置放入 R 即可。

思路2：可行解存在条件不变，考虑将数组中的每一个数看作点。边有两种——值边和位边。

值边只连接两个值相同的点：想象着将每种值相同的位置存入数组 $p[]$ （长度必定为偶数），将 $p[2*i-1]$ 和 $p[2*i]$ 之间连边。

位边只连接在数组中位置相邻的两个点：设这个数组为 $a[]$ （长度必定为偶数），将 $a[2*i-1]$ 和 $a[2*i]$ 之间连边。

这是一张二分图，因为不存在奇环。因为如果有奇环，必然有两个值边或两个位边相邻（有公共端点）。而每个点有且仅有一条值边和一条位边连出去，矛盾，原命题得证。

最后，我们将这个二分图黑白染色，黑的去 L ，白的去 R 即可。

差分约束系统

定义

差分约束系统 是一种特殊的 n 元一次不等式组，它包含 n 个变量 $x_1, x_2, \dots, x_n$ 以及 m 个约束条件，每个约束条件是由两个其中的变量做差构成的，形如 $x_i - x_j \leq c_k$ ，其中 $1 \leq i, j \leq n, i \neq j, 1 \leq k \leq m$ 并且 c_k 是常数。我们要解决的问题是：求一组解，使得所有的约束条件得到满足，否则判断出无解。

差分约束系统中的每个约束条件 $x_i - x_j \leq c_k$ 都可以变形为 $x_i \leq x_j + c_k$ ，这与单源最短路中的三角形不等式 $dis(y) \leq dis(x) + z$ 非常相似。因此，我们可以把每个变量 x_i 看做图中的一个结点，对于每个约束条件 $x_i - x_j \leq c_k$ ，从结点 j 向结点 i 连一条长度为 c_k 的有向边。

注意到，如果 $a_1, a_2, \dots, a_n$ 是该差分约束系统的一组解，那么对于任意的常数 d ， $a_1 + d, a_2 + d, \dots, a_n + d$ 显然也是该差分约束系统的一组解，因为这样做差后 d 刚好被消掉。

实现

设立一个超级源点 $S = 0, dis(0) = 0$ ，并向每一个点连一条权重为 0 的边，跑单源最短路，若图中存在负环，则给定的差分约束系统无解，否则， $x_i = dis(i)$ 为该差分约束系统的一组解。具体实现多为 Bellman-Ford，时间复杂度为 $O(nm)$ 。

例题

P1993 小 K 的农场

题意：求解差分约束系统，有 m 条约束条件，每条都为形如 $x_i - x_j \leq c_k$ ， $x_i - x_j \geq c_k$ 或 $x_i = x_j$ 的形式，判断该差分约束系统有没有解。

$$(x_i - x_j \leq c_k) \rightarrow (x_i - x_j \leq c_k)$$

$$(x_i - x_j \geq c_k) \rightarrow (x_j - x_i \leq -c_k)$$

$$(x_i = x_j) \rightarrow (x_i - x_j \leq 0, x_j - x_i \leq 0)$$

P4926 倍杀测量者

考虑 $\frac{x_i}{x_j} \leq c_k$ 的变形方法：

$$\left(\frac{x_i}{x_j} \leq c_k\right) \rightarrow \left(\log\left(\frac{x_i}{x_j}\right) \leq \log c_k\right) \rightarrow (\log x_i - \log x_j \leq \log c_k)$$

树上问题

定义

一个没有固定根结点的树称为 **无根树** (unrooted tree)。无根树有几种等价的形式化定义：

- 有 n 个结点， $n - 1$ 条边的连通无向图
- 无向无环的连通图
- 任意两个结点之间有且仅有一条简单路径的无向图
- 任何边均为桥的连通图
- 没有圈，且在任意不同两点间添加一条边之后所得图含唯一的一个圈的图

在无根树的基础上，指定一个结点称为 **根**，则形成一棵 **有根树** (rooted tree)。

适用于所有树

森林 (forest)：每个连通分量（连通块）都是树的图，一棵树也是森林。

生成树 (spanning tree)：一个连通无向图的生成子图，同时要求是树。也即在图的边集中选择 $n - 1$ 条，将所有顶点连通。

无根树的叶结点 (leaf node)：度数不超过 1 的结点。

有根树的叶结点 (leaf node)：没有子结点的结点。

适用于有根树

父亲 (parent node)：对于除根以外的每个结点，定义为从该结点到根路径上的第二个结点。根结点没有父结点。

祖先 (ancestor)：一个结点到根结点的路径上，除了它本身外的结点。根结点的祖先集合为空。

子结点 (child node)：如果 u 是 v 的父亲，那么 v 是 u 的子结点。
子结点的顺序一般不加以区分，二叉树是一个例外。

结点的深度 (depth)：到根结点的路径上的边数。

树的高度 (height)：所有结点的深度的最大值。

兄弟 (sibling)：同一个父亲的多个子结点互为兄弟。

后代 (descendant)：子结点和子结点的后代。

或者理解成：如果 u 是 v 的祖先，那么 v 是 u 的后代。

子树 (subtree)：删掉与父亲相连的边后，该结点所在的子图。

特殊的树

- **链 (chain/path graph)**：满足与任一结点相连的边不超过 2 条的树称为链。
- **菊花/星星 (star)**：满足存在 u 使得所有除 u 以外结点均与 u 相连的树称为菊花。
- **有根二叉树 (rooted binary tree)**：每个结点最多只有两个儿子（子结点）的有根树称为二叉树。常常对两个子结点的顺序加以区分，分别称之为左子结点和右子结点。大多数情况下，**二叉树** 一词均指有根二叉树。
- **完整二叉树 (full/proper binary tree)**：每个结点的子结点数量均为 0 或者 2 的二叉树。换言之，每个结点或者是树叶，或者左右子树均非空。
- **完全二叉树 (complete binary tree)**：只有最下面两层结点的度数可以小于 2，且最下面一层的结点都集中在该层最左边的连续位置上。
- **完美二叉树 (perfect binary tree)**：所有叶结点的深度均相同的二叉树称为完美二叉树。

树的存储

记录父节点

仅使用数组 `fa[]` 来记录每个结点的父节点。

难以进行自顶向下的遍历，多用于自底向上的递推问题。

邻接表

与图的邻接表表示方法类似，在此不赘述。

左孩子右兄弟表示法

仅对于有根树。

对每个结点的子节点指定一个顺序序列。

对于每个节点记录两个值：其 **第一个子节点** `child[u]` 和其 **下一个兄弟节点** `sib[u]`。若没有子节点，则 `child[u]` 为 -1 ；若该节点为其父节点的最后一个子节点，则 `sib[u]` 为 -1 。

树的遍历

树上DFS

先访问根节点，然后分别访问根节点每个儿子的子树。

可以用来求出每个节点的深度、父亲、子树大小等。

```

1 void dfs(int u,int pre)
2 {
3     fa[u]=pre;
4     dep[u]=dep[pre]+1;
5     siz[u]=1;
6     for(auto v:vec[u])if(v!=pre)
7     {
8         dfs(v,u);
9         siz[u]+=siz[v];
10    }
11 }

```

二叉树上DFS

先序遍历

按照 **根，左儿子，右儿子** 的顺序遍历二叉树。

```

1 void preTrav(int u)
2 {
3     cout<<u<<" ";
4     if(~lch[u])preTrav(lch[u]);
5     if(~rch[u])preTrav(rch[u]);
6 }

```

中序遍历

按照 **左儿子，根，右儿子** 的顺序遍历二叉树。

```

1 void midTrav(int u)
2 {
3     if(~lch[u])preTrav(lch[u]);
4     cout<<u<<" ";
5     if(~rch[u])preTrav(rch[u]);
6 }

```

后序遍历

按照 **左儿子，右儿子，根** 的顺序遍历二叉树。

```

1 void lastTrav(int u)
2 {
3     if(~lch[u])preTrav(lch[u]);
4     if(~rch[u])preTrav(rch[u]);
5     cout<<u<<" ";
6 }

```

反推

已知中序遍历序列和另外一个序列可以求第三个序列。

1. 前序的第一个是 root，后序的最后一个为 root。
2. 先确定根节点，然后根据中序遍历，在根左边的为左子树，根右边的为右子树。
3. 对于每一个子树可以看成是一个全新的树，仍然遵循上面的规律。

树上BFS

从树根开始，严格按照层次来访问节点。

BFS 过程中也可以顺便求出各个节点的深度和父亲节点。

树的直径

定义

树上任意两节点之间最长的简单路径即为 **树的直径**。

1.DFS

以下实现和定理均要求树中不存在负边权。

首先从任意节点 u 开始进行第一次 DFS，到达距离其最远的节点，记为 v ，然后再从 v 开始做第二次 DFS，到达距离 v 最远的节点，记为 w ，则 $dis(v, w)$ 即为树的直径。

显然，如果第一次 DFS 到达的节点 v 是直径的一端，那么第二次 DFS 到达的节点 w 一定是直径的一端。我们只需证明在任意情况下， w 必为直径的一端。

定理：在一棵树上，从任意节点开始进行一次 DFS，到达的距离其最远的节点 x 必为直径的一端。

实现：

```
1  vector<ll>vec[N];
2  ll dis[N],k;
3  void dfs(ll u,ll fa)
4  {
5      for(auto v:vec[u])
6      {
7          if(v==fa)continue;
8          dis[v]=dis[u]+1;
9          if(dis[v]>dis[k])k=v;
10         dfs(v,u);
11     }
12 }
13 int main()
14 {
15     int n;
16     cin>>n;
17     for(int i=2;i<=n;++i)
18     {
19         ll u,v;
20         cin>>u>>v;
21         vec[u].push_back(v);
22         vec[v].push_back(u);
23     }
24     k=1;
25     dfs(1,0);
26     dis[k]=0;
27     dfs(k,0);
28     cout<<dis[k];
29 }
```

2.树形DP

我们记录当 1 为树的根时，每个节点作为子树的根向下，所能延伸的最远距离 d_1 ，和次远距离 d_2 ，那么直径就是所有 $d_1 + d_2$ 的最大值。

树形 DP 可以在存在负权边的情况下求解出树的直径。

实现：

```
1  vector<ll>vec[N];
2  ll maxn=0;
3  ll d1[N],d2[N];
4  void dfs(ll u,ll fa)
5  {
6      d1[u]=d2[u]=0;
7      for(auto v:vec[u])
8      {
9          if(v==fa)continue;
10         dfs(v,u);
11         if(d1[v]+1>d1[u])
12         {
13             d2[u]=d1[u];
14             d1[u]=d1[v]+1;
15         }
16         else if(d1[v]+1>d2[u])
17         {
18             d2[u]=d1[v]+1;
19         }
20     }
21     maxn=max(maxn,d1[u]+d2[u]);
22 }
23 int main()
24 {
25     int n;
26     cin>>n;
27     for(int i=2;i<=n;++i)
28     {
29         ll u,v;
30         cin>>u>>v;
31         vec[u].push_back(v);
32         vec[v].push_back(u);
33     }
34     dfs(1,0);
35     cout<<maxn;
36 }
```

性质

若树上所有边边权均为正，则树的所有直径中点重合。

树的重心

定义

对于树上的每一个点，计算其所有子树中最大的子树节点数，这个值最小的点就是这棵树的重心。

无根树以树的重心为根时，所有子树的大小都不超过整棵树大小的一半。

树中所有点到某个点的距离和中，到重心的距离和是最小的；如果有两个重心，那么到它们的距离和一样。

把两棵树通过一条边相连得到一棵新的树，那么新的树的重心在连接原来两棵树的重心的路径上。

在一棵树上添加或删除一个叶子，那么它的重心最多只移动一条边的距离。

DFS

在 DFS 中计算每个子树的大小，记录“向下”的子树的最大大小，利用总点数 - 当前子树（这里的子树指有根树的子树）的大小得到“向上”的子树的大小，根据定义就可以找到重心了。

实现：

```
1  vector<ll>vec[N];
2  ll siz[N],son[N],r1,r2;//r1,r2可能的重心
3  void dfs(ll u,ll fa)//siz:以i为根节点的子树节点个数,son:删去节点i后剩下连通分量最大子
   树节点个数
4  {
5      siz[u]=1;
6      son[u]=0;
7      for(auto v:vec[u])
8      {
9          if(v==fa)continue;
10         dfs(v,u);
11         siz[u]+=siz[v];
12         son[u]=max(son[u],siz[v]);
13     }
14     son[u]=max(son[u],n-siz[u]);
15     if((son[u]<=1)<=n)r2=r1,r1=u;
16 }
```

LCA

定义

最近公共祖先简称 **LCA (Lowest Common Ancestor)**。两个节点的最近公共祖先，就是这两个点的公共祖先里面，离根最远的那个。为了方便，我们记某点集 $S = v_1, v_2, \dots, v_n$ 的最近公共祖先为 $\text{LCA}(v_1, v_2, \dots, v_n)$ 或 $\text{LCA}(S)$ 。

1. $\text{LCA}(u) = u$;
2. u 是 v 的祖先，当且仅当 $\text{LCA}(u, v) = u$;
3. 如果 u 不为 v 的祖先并且 v 不为 u 的祖先，那么 u, v 分别处于 $\text{LCA}(u, v)$ 的两棵不同子树中；
4. 前序遍历中， $\text{LCA}(S)$ 出现在所有 S 中元素之前，后序遍历中 $\text{LCA}(S)$ 则出现在所有 S 中元素之后；
5. 两点集并的最近公共祖先为两点集分别的最近公共祖先的最近公共祖先，即 $\text{LCA}(A \cup B) = \text{LCA}(\text{LCA}(A), \text{LCA}(B))$ ；
6. 两点的最近公共祖先必定处在树上两点间的最短路上；
7. $d(u, v) = h(u) + h(v) - 2h(\text{LCA}(u, v))$ ，其中 d 是树上两点间的距离， h 代表点到树根的距离。

朴素算法

每次找深度比较大的那个点，让它向上跳到它的父节点，深度相同时，共同向上跳转，最后一定会相遇。

预处理时需要 DFS 整棵树，时间复杂度为 $O(n)$ ，单次查询时间复杂度为 $\Theta(n)$ 。

倍增算法

通过预处理 `fa[][]` 数组，使得游标快速移动，大幅减少了游标跳转次数。其中 `fa[x][i]` 表示点 x 的第 2^i 个祖先。`fa[][]` 数组可以通过 DFS 预处理出来。

在调整游标的第一阶段中，我们要将 u, v 两点跳转到同一深度。我们可以计算出两点的深度之差，设其为 Δd 。通过将其进行二进制拆分，我们将 Δd 次游标跳转优化为 Δd 的二进制表示所含 1 的个数次游标跳转。在第二阶段中，我们从最大的 i 开始循环尝试，一直尝试到 0，如果 `fa[u][i] != fa[v][i]`，则 $u \leftarrow fa[u][i]$ ， $v \leftarrow fa[v][i]$ ，那么最后的 LCA 为 `fa[u][0]`。

倍增算法的预处理时间复杂度为 $O(n \log n)$ ，单次查询时间复杂度为 $O(\log n)$ 。

实现：

```
1  ll fa[N][22], dep[N], lg[N], siz[N];
2  ll root, n;
3  vector<ll> vec[N];
4  void init()
5  {
6      for(int i=1; i<=n; ++i) lg[i]=lg[i-1]+(1<<lg[i-1]==i);
7      return ;
8  }
9  void dfs(ll u, ll pre)
10 {
11     fa[u][0]=pre;
12     siz[u]=1;
13     dep[u]=dep[pre]+1;
14     for(int i=1; i<=lg[dep[u]]; ++i) fa[u][i]=fa[fa[u][i-1]][i-1];
15     for(auto v:vec[u]) if(v!=pre)
16     {
17         dfs(v, u);
18         siz[u]+=siz[v];
19     }
20 }
21 ll LCA(ll x, ll y)
22 {
23     if(dep[x]<dep[y]) swap(x, y);
24     while(dep[x]>dep[y]) x=fa[x][lg[dep[x]]-dep[y]-1];
25     if(x==y) return x;
26     for(int i=lg[dep[x]]-1; i>=0; --i) if(fa[x][i]!=fa[y][i]) x=fa[x][i], y=fa[y][i];
27     return fa[x][0];
28 }
29 ll Dis(ll x, ll y)
30 {
31     return dep[x]+dep[y]-2*dep[LCA(x, y)];
32 }
33 int main()
34 {
35     cin>>n;
36     ll root=1;
```

```

37     init();
38     for(int i=2;i<=n;++i)
39     {
40         ll u,v;
41         cin>>u>>v;
42         vec[u].push_back(v);
43         vec[v].push_back(u);
44     }
45     dfs(root,0);
46 }

```

欧拉序

对一棵树进行 DFS，无论是第一次访问还是回溯，每次到达一个结点时都将编号记录下来，可以得到一个长度为 $2n - 1$ 的序列，这个序列被称作这棵树的欧拉序列。

在下文中，把结点 u 在欧拉序列中第一次出现的位置编号记为 $pos(u)$ （也称作节点 u 的欧拉序），把欧拉序列本身记作 $E[1..2n - 1]$ 。

有了欧拉序列，LCA 问题可以在线性时间内转化为 RMQ 问题，即：

$$pos(LCA(u, v)) = \min\{pos(k) | k \in E[pos(u)..pos(v)]\}$$

这个等式不难理解：从 u 走到 v 的过程中一定会经过 $LCA(u, v)$ ，但不会经过 $LCA(u, v)$ 的祖先。因此，从 u 走到 v 的过程中经过的欧拉序最小的结点就是 $LCA(u, v)$ 。

用 DFS 计算欧拉序列的时间复杂度是 $O(n)$ ，且欧拉序列的长度也是 $O(n)$ ，所以 LCA 问题可以在 $O(n)$ 的时间内转化成等规模的 RMQ 问题。

当我们需要查询某点对 (u, v) 的 LCA 时，查询区间 $[\min\{pos[u], pos[v]\}, \max\{pos[u], pos[v]\}]$ 上最小值的所代表的节点即可。

若使用 ST 表来解决 RMQ 问题，那么该算法不支持在线修改，预处理的时间复杂度为 $O(n \log n)$ ，每次查询 LCA 的时间复杂度为 $O(1)$ 。

实现：

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  const int N=5e5+5;
4  vector<int>dfn,pos,mp;
5  vector<int>vec[N];
6  void dfs(int u,int pre)
7  {
8      dfn.push_back(u);
9      pos[u]=dfn.size()-1;
10     mp[dfn.size()-1]=u;
11     for(auto v:vec[u])if(v!=pre)
12     {
13         dfs(v,u);
14         dfn.push_back(u);
15     }
16 }
17 template <typename T>
18 class SparseTable
19 {
20     using VT = vector<T>;
21     using VVT = vector<VT>;
22     using func_type = function<T(const T &, const T &)>;

```

```

23
24     VT V;
25     VVT ST;
26
27     static T default_func(const T &t1, const T &t2) { return min(t1, t2); }
28
29     func_type op;
30
31 public:
32     SparseTable(const vector<T> &v, func_type _func = default_func) {
33         v = v;
34         op = _func;
35         int len = v.size(), l1 = ceil(log2(len)) + 1;
36         ST.assign(len, VT(l1, 0));
37         for (int i = 0; i < len; ++i) {
38             ST[i][0] = v[i];
39         }
40         for (int j = 1; j < l1; ++j) {
41             int pj = (1 << (j - 1));
42             for (int i = 0; i + pj < len; ++i) {
43                 ST[i][j] = op(ST[i][j - 1], ST[i + (1 << (j - 1))][j - 1]);
44             }
45         }
46     }
47
48     T query(int l, int r) {
49         if (l == r) return v[l];
50         int lt = r - l + 1;
51         int q = ceil(log2(lt)) - 1;
52         return op(ST[l][q], ST[r - (1 << q) + 1][q]);
53     }
54 };
55 int LCA(int x, int y, SparseTable<int> &st)
56 {
57     x = pos[x], y = pos[y];
58     if (x == y) return mp[x];
59     return mp[st.query(min(x, y), max(x, y))];
60 }
61 int main()
62 {
63     ios::sync_with_stdio(0);
64     cin.tie(0);
65     int n, m, s;
66     cin >> n >> m >> s;
67     --s;
68     pos.resize(n);
69     mp.resize(2 * n);
70     for (int i = 2; i <= n; ++i)
71     {
72         int u, v;
73         cin >> u >> v;
74         --u, --v;
75         vec[u].push_back(v);
76         vec[v].push_back(u);
77     }
78     dfs(s, -1);
79     for (auto &x: dfn) x = pos[x];
80     SparseTable<int> st(dfn);

```

```

81     for(int i=1;i<=m;++i)
82     {
83         int x,y;
84         cin>>x>>y;
85         --x,--y;
86         cout<<LCA(x,y,st)+1<<"\n";
87     }
88 }

```

树上启发式合并

常见实现

主要思想在于 **保留较多的信息**，把其他零散的信息暴力地往上面合并。

并查集的按秩合并：

```

1 void merge(int x,int y)
2 {
3     int fx=find(x),fy=find(y);
4     if(siz[fx]<siz[fy])swap(fx,fy);
5     fa[fy]=fx;
6     siz[fx]+=siz[fy];
7 }

```

对于两个大小不一样的集合，我们将小的集合合并到大的集合中，而不是将大的集合合并到小的集合中。

集合的大小可以认为是集合的高度，而我们将集合高度小的并到高度大的显然有助于我们找到父亲。实际上按秩合并的复杂度是 $O(\log n)$ 的。

例题

CF600E Lomsat gelral

题意：有一棵 n 个结点的以 1 号结点为根的有根树。

每个结点都有一个颜色 c_i 。

如果一种颜色在以 x 为根的子树内出现次数最多，称其在以 x 为根的子树中占主导地位。显然，同一子树中可能有多种颜色占主导地位。

对于每一个 $i \in [1, n]$ ，求出以 i 为根的子树中，占主导地位的颜色的编号和。

$\sum n \leq 10^5$

思路：考虑遍历整棵树，并用一个数组 `cnt` 记录每种颜色出现几次。

但每做完一棵子树就需要清空 `cnt`，以免对其兄弟造成影响。

而这样做它的祖先时就要把它重新搜一遍，显然很慢。

考虑对于每个节点 v ，选择所含节点最多的一棵子树称为 **重儿子**，作为最后遍历的子树不清空，做完后把其结果直接加入 v 的答案中。

由于对于每个节点，它被计算的次数就是它到根节点路径上的轻边（连到轻儿子的边）数

我们只需算出轻边有几条

由于每从一个深度为 d 的节点沿一条轻边走到深度 $(d - 1)$ 的节点，子树大小就至少 ≥ 2 （这是因为有一个重儿子 $\geq$ 当前子树）

所以最多只有 $\log n$ 条轻边，复杂度为 $O(n \log n)$ 。

CF570D Tree Requests

题意：给定一个以 1 为根的 n 个结点的树，每个点上有一个字母（a-z），每个点的深度定义为该节点到 1 号结点路径上的点数。每次询问 a, b 查询以 a 为根的子树内深度为 b 的结点上的字母重新排列之后是否能构成回文串。

思路：对于一个节点，统计它的子树中所有深度的节点的颜色情况，询问的时候只要检查这个节点的子树中指定深度的节点颜色出现次数是否有 ≤ 1 个奇数即可。

考虑优化空间复杂度，把所有询问离线下来，根据节点编号分类（把关于 u 节点的询问挂到 u 上，DFS 的时候一起处理）。

接下来我们每处理一个节点的时候只需要把前面统计的东西清空，全部暴力统计一遍。

考虑优化时间复杂度，由于在清空之前的统计数据的时候造成了很大的浪费，我们每次处理的时候把重儿子的信息留着，把轻儿子的信息删去，最后再暴力地把轻儿子合并到留下的信息里面。

由于每个轻儿子最多会被合并 $\log n$ 次，所以总时间复杂度为 $O(n \log n)$ 。

CF1709E XOR Tree

题意：有一棵无根树，点数为 n ，每个点有个点权 a_i ，定义一条路径 $P(u, v)$ 的权值为经过的所有点的点权的异或和。定义一棵树是合法的，当且仅当树上所有简单路径（只经过每个点一次的路径）的权值都不为 0。

你可以对权值进行修改，可以改成任意正整数，问最少修改多少次才能让这棵树合法。

$$\sum n \leq 2 \times 10^5, 1 \leq a_i \leq 2^{30}$$

思路：记 dis_x 为从根到点 x 的异或和，考虑两个点 (u, v) 之间简单路径和为 $dis_u \oplus dis_v \oplus a_{LCA(u,v)}$ （消去了 $dis_{LCA(u,v)}$ 的部分）。

接下来我们考虑路径异或和为 0 的含义，我们发现，一对 (u, v) 异或和为 0 当且仅当他们的异或和等于 $a_{LCA(u,v)}$ 。

对于每个点 x 来说，把他作为 $LCA(u, v)$ ，那么子树内如果存在一对点 (u, v) 满足 $dis_u \oplus a_x = dis_v$ ，我们就必须将 x 的点权进行修改。

考虑启发式合并，用 set 维护每个点子树里的点权。先进行一遍 dfs 找出重儿子，以及 dis 的预处理。第二遍 dfs 将重儿子的集合换上来，对于每个轻儿子，再对其进行查询，并且合并集合。若是发现当前子树内有需要的点，直接让答案加一，清空当前集合（已经通过修改 a_x 使得当前子树内不再拥有可以满足上述条件的点对）。

时间复杂度 $O(n \log n^2)$ 。

分层图

如：有 k 次零代价通过一条路径，求总的最小花费。对于这种题目，我们可以采用 DP 相关的思想，设 $dis(i, j)$ 表示当前从起点 i 号结点，使用了 j 次免费通行权限后的最短路径。显然， dis 数组可以这么转移： $dis(i, j) = \min\{\min\{dis(from, j-1)\}, \min\{dis(from, j) + w\}\}$

其中， $from$ 表示 i 的父亲节点， w 表示当前所走的边的边权。当 $j-1 \geq k$ 时， $dis(from, j) = +\infty$ 。

事实上，这个 DP 就相当于把每个结点拆分成了 $k+1$ 个结点，每个新结点代表使用不同多次免费通行后到达的原图结点。换句话说， u_i 就是每个结点 u 表示使用 i 次免费通行权限后到达 u 结点。

乱做

CF1737D Ela and the Wiring Wizard

题意：有一个包含 n 个点和 m 条边的无向图，边权为 w_i 。

对于任意三点 x, y, z （ x 和 z 可以相同），若 x 与 y 之间有边 i ，且 y 与 z 有边 j ，那么他可以用 w_i 断开边 i ，并且在 x 与 z 之间连一条权值为 w_i 的边。可以改任意条边，也可以不改。

求从 1 号点走到 n 号点的最短路。

保证输入的图为联通图，无自环，但可能有重边。

$$\sum n \leq 500, \sum m \leq 500^2$$

思路：考虑肯定存在一种最优的走法是让某一条边变成连接 1 和 n 然后走这条边。因为如果这不是最优的，那么这条路径上必然存在另一条更短的边，那我们换成用这条更短的边走就行（用这条短边肯定好过用原来那条长边）。

于是问题就变成让一条边两个端点在图上走，可以走自己，（分别到达 1 和 n 的步数 + 1）乘以边权的最小值。

设两点 u, v 最短路为 $dis(u, v)$ ，这里的图是原图把所有边权改成 1 后得到的图。然后分类讨论一下每条边 (x, y) ：

- 两个端点分别直接走到 1 和 n ，即 $\min\{dis(x, 1) + dis(y, n), dis(x, n) + dis(y, 1)\}$
- 枚举中间点 k ，一个端点走到某个中间点 k ，然后另一个端点走自己这条边使得两个点都在 k 上然后分别走到 1 和 n ，即
$$\min\{dis(x, k) + 1 + dis(k, 1) + dis(k, n), dis(y, k) + 1 + dis(k, 1) + dis(k, n)\}$$

Floyd 求最短路，然后枚举每条边取 min 即可。

```
1  int main()
2  {
3      ios::sync_with_stdio(false);
4      cin.tie(nullptr), cout.tie(nullptr);
5      //IO
6      cin >> t;
7      while(t--)
8      {
9          cin >> n >> m;
10         vector<vector<ll>>> f(n+1, vector<ll>(n+1, 1e18));
11         vector<ll> uu(m+1), vv(m+1), ww(m+1);
12         rep(i, 1, m)
13         {
14             ll u, v, w;
15             cin >> u >> v >> w;
16             uu[i] = u, vv[i] = v, ww[i] = w;
17             f[u][v] = 1LL;
18             f[v][u] = 1LL;
19         }
20         rep(i, 1, n) f[i][i] = 0;
21         rep(k, 1, n) rep(i, 1, n) rep(j, 1, n) f[i][j] = min(f[i][j], f[i][k] + f[k][j]);
22         ll ans = 1e18;
23         rep(i, 1, m)
24         {
25             ll len = min(f[1][uu[i]] + f[n][vv[i]], f[1][vv[i]] + f[n][uu[i]]);
26             ans = min(ans, ww[i] * (1 + len));
27             rep(k, 1, n)
28             {
29                 ans = min(ans, ww[i] * (f[k][uu[i]] + 1 + f[1][k] + f[k][n] + 1));
```

```

30         ans=min(ans,ww[i]*(f[k][vv[i]]+1+f[1][k]+f[k][n]+1));
31     }
32 }
33     cout<<ans<<"\n";
34 }
35 }

```

CF1749E Cactus Wall

题意：为了抵御怪物的袭击，Monocarp 决定利用他的院子构建一座仙人掌屏障。院子可以看作 n 行 m 列的网格，每个格子内最多可以种一颗仙人掌。

怪物可以从第一行任意一个没有仙人掌的格子进入院子，它每次能走到一个**四联通相邻**（即两个格子有公共边）且没有仙人掌的格子。如果怪物怎么走都无法到达最后一行的某个没有仙人掌的格子，那么院子里就成功构建了一座屏障。

但如果仙人掌种植过密，就可能因为缺乏养分凋零而失去作用，因此**任意两个四联通相邻的格子不能同时种上仙人掌**。

现在可能有些格子已经种上了仙人掌。Monocarp 想要知道在**不铲除已有仙人掌**的前提下，至少要再种多少颗仙人掌才能构成一座屏障。

$$\sum nm \leq 4 \times 10^5$$

思路：考虑 BFS 仙人掌的种植路线，仙人掌四联通格子不能同时种，所以 BFS 的方向只能是斜着的四个方向，然后题目要求种植的仙人掌要最少，想到这条隔断上方的仙人掌路线要尽可能经过原本的仙人掌，将到仙人掌格子的边权设为 0，到要种植仙人掌的格子的边权设为 1。使用 01BFS 求解即可。

```

1  bool check(ll x,ll y,vector<string> &s)
2  {
3      if(min(x,y)<1||x>n||y>m)return 0;
4      rep(i,0,3)
5      {
6          ll xx=x+dir[i][0];
7          ll yy=y+dir[i][1];
8          if(s[xx][yy]=='#')return 0;
9      }
10     return 1;
11 }
12 int main()
13 {
14     ios::sync_with_stdio(false);
15     cin.tie(nullptr),cout.tie(nullptr);
16     //IO
17     cin>>t;
18     while(t--)
19     {
20         cin>>n>>m;
21         vector<string>s(n+2);
22         vector<vector<ll>>dp(n+1,vector<ll>(m+1,-1));
23         s[0]=s[n+1]=string(m+1,'$');
24         rep(i,1,n)
25         {
26             cin>>s[i];
27             s[i]+='$';
28             s[i]="$"+s[i];
29         }
30         deque<pll>q;

```

```

31     rep(i,1,n)
32     {
33         if(s[i][1]=='#')
34         {
35             dp[i][1]=0;
36             q.push_front({i,1});
37         }
38         else if(check(i,1,s))
39         {
40             dp[i][1]=1;
41             q.push_back({i,1});
42         }
43     }
44     map<p11,p11>pre;
45     while(q.size())
46     {
47         int ddr[4][2]={1,1},{1,-1},{-1,-1},{-1,1};
48         p11 u=q.front();
49         11 x=u.ft,y=u.sd;
50         q.pop_front();
51         rep(i,0,3)
52         {
53             11 xx=x+ddr[i][0];
54             11 yy=y+ddr[i][1];
55             if(!check(xx,yy,s)||dp[xx][yy]==-1)continue;
56             if(s[xx][yy]=='#')
57             {
58                 dp[xx][yy]=dp[x][y];
59                 pre[{xx,yy}]=x,y;
60                 q.push_front({xx,yy});
61             }
62             else
63             {
64                 dp[xx][yy]=dp[x][y]+1;
65                 pre[{xx,yy}]=x,y;
66                 q.push_back({xx,yy});
67             }
68         }
69     }
70     11 mn=1e18;
71     p11 used={-1,-1};
72     rep(i,1,n)
73     {
74         if(dp[i][m]==-1&&dp[i][m]<mn)
75         {
76             mn=dp[i][m];
77             used={i,m};
78         }
79     }
80     if(mn==1e18)
81     {
82         no;
83         continue;
84     }
85     yes;
86     while(1)
87     {
88         s[used.ft][used.sd]='#';

```

```

89         if(!pre.count(used))break;
90         used=pre[used];
91     }
92     rep(i,1,n)
93     {
94         rep(j,1,m)cout<<s[i][j];
95         cout<<"\n";
96     }
97 }
98 }

```

CF1650G Counting Shortcuts

题意：给定一张无向图，求出与 s, t 之间的与最短路长度差不超过 1 的路径条数对 $10^9 + 7$ 取模。

$$\sum n, \sum m \leq 2 \times 10^5$$

思路：寻找与最短路长度相差不超过 1 的路径数，考虑到无边权，转化为“寻找次短路径数”。对于最短路，考虑 Dijkstra 算法，有最短路更新最短路，更新则记录原最短路为次短路。对于每条路径，判断长度是否等于次短路，等于便更新答案即可。

```

1  struct edge
2  {
3      int to,next,val;
4  }dat[M];
5  void Dijkstra(int s)
6  {
7      priority_queue<pa,vector<pa>,greater<pa> >q;
8      vector<ll>inq((n+1)<<1,0);
9      rep(i,0,n)dis[i]=INF;
10     dis[s]=0;num[s]=1;q.push(make_pair(dis[s],s));
11     while(!q.empty())
12     {
13         ll x=q.top().second,w=q.top().first,cnt;q.pop();
14         if(inq[x]) continue;
15         inq[x]=1;
16         if(x>n)x-=n,cnt=num1[x];
17         else cnt=num[x];
18         for(ll i=h[x];i;i=dat[i].next)
19         {
20             ll y=dat[i].to;
21             if(dis[y]>w+dat[i].val)
22             {
23                 dis1[y]=dis[y];num1[y]=num[y];
24                 dis[y]=w+dat[i].val;num[y]=cnt;
25                 q.push(make_pair(dis[y],y));
26                 q.push(make_pair(dis1[y],y+n));
27                 continue;
28             }
29             if(dis[y]==w+dat[i].val)
30             {
31                 num[y]=(num[y]+cnt)%P;
32                 continue;
33             }
34             if(dis1[y]>w+dat[i].val)
35             {
36                 dis1[y]=w+dat[i].val;num1[y]=cnt;

```

```

37         q.push(make_pair(dis1[y],y+n));
38         continue;
39     }
40     if(dis1[y]==w+dat[i].val)
41     {
42         num1[y]=(num1[y]+cnt)%P;
43         continue;
44     }
45 }
46 }
47 }
48 int main()
49 {
50     ios::sync_with_stdio(0);
51     cin.tie(0);
52     cin>>T;
53     while(T--)
54     {
55         ll s,t;
56         cin>>n>>m>>s>>t;
57         num0=0;
58         while(m--)
59         {
60             ll x,y;cin>>x>>y;
61             dat[++num0].to=y;dat[num0].next=h[x];h[x]=num0;dat[num0].val=1;
62             dat[++num0].to=x;dat[num0].next=h[y];h[y]=num0;dat[num0].val=1;
63         }
64         Dijkstra(s);
65         if(dis1[t]-1==dis[t]) num[t]=(num[t]+num1[t])%P;
66         cout<<num[t]<<'\n';
67         rep(i,0,n)h[i]=0;
68     }
69 }

```